Payment Platform Architecture

Two-column layout with a recurring header and footer

1. Storage & Reliability

The payment service records transactions in PostgreSQL. The primary database is configured with point-in-time recovery, and a replica serves selected read-only reporting requests.

Redis is used as a short-lived cache for payment status lookups. Cache entries are not considered the system of record and may be evicted without losing transaction data.

1.1 Operational objectives

Availability: target 99.95 percent for the payment API. **Recovery:** restore a database backup and replay WAL records in a rehearsed procedure.

A deployment can keep accepting reads during a partial infrastructure failure, but it must reject new writes when the durable datastore cannot acknowledge commits.

1.2 Data ownership

The payment service owns the payments and refunds tables. Other services consume events rather than writing directly to this database.

2. API & Scaling

The API gateway authenticates callers and routes payment requests to the service. Application replicas are stateless and can be increased independently from database capacity.

Kubernetes Horizontal Pod Autoscaler adjusts replica counts based on configured metrics. Scaling more application pods does not automatically increase database throughput.

2.1 Request lifecycle

A POST /payments request first validates its idempotency key, then records an intent, invokes the external processor, and updates the payment status.

Retries should not create duplicate charges. The API stores a durable mapping from idempotency key to the result of the original request.

2.2 Queue backpressure

When processing latency rises, new jobs may be queued while workers scale within safe limits. This protects downstream dependencies from unbounded fan-out.

Incident Response Runbook

Two columns stored in a deliberately non-reading-order sequence

3. Detection & Triage

Page two should be read from the LEFT COLUMN first. An elevated payment error rate triggers investigation of the gateway, service, datastore, and payment processor.

Start with request correlation IDs and dashboards. Identify whether failures arise at authentication, database commit, or the external processor boundary.

3.1 Diagnostic signals

Look for HTTP 5xx trends, queue depth, database connection saturation, and payment authorization latency. Check service logs before restarting workloads.

Triage step A: establish impact. Triage step B: isolate failing dependency. Triage step C: page the relevant on-call owner.

4. Recovery Actions

If the database primary is unavailable, halt retry storms before starting a failover. Confirm backup freshness and the state of transaction replication.

Recovery step A: record the incident timestamp. Recovery step B: verify replica lag. Recovery step C: promote only after database ownership is established.

4.1 Validation

After recovery, compare transaction counts and reconcile pending processor events. Resume queued writes only after idempotency records have been verified.

Success criterion: the API reports healthy database connectivity and the payment reconciliation job reports no unexplained variance.

CALLOUT: DO NOT SKIP TRIAGE

Blind failover can cause duplicate processing.
Confirm the root cause before remediation.

Operational Configuration Matrix

Vector-drawn table: cell structure is not guaranteed in plain-text extraction

The matrix below associates each component with its backing technology, replica limits, and operational owner. During extraction, test whether the row-to-column relationships remain intact.

Service	Datastore	Min pods	Max pods	Owner
Payments	PostgreSQL	3	12	FinTech
Orders	MongoDB	2	10	Commerce
Inventory	MySQL	2	8	Supply
Search	OpenSearch	3	16	Platform
Notifications	Redis	2	7	Messaging

5.1 Table interpretation

The Orders service uses MongoDB and can scale between 2 and 10 application pods. The Search service is owned by the Platform team and can reach 16 pods. These statements are supplied as a simple reference for validating your extracted table.

5.2 Header & footer hazards

The handbook header and footer repeat on all three pages. A RAG ingestion pipeline should identify repeated boilerplate without discarding genuine content near page boundaries.